



DeFuzz: LLM 驱动的编译器软件防御机制模糊测试

硕士学位论文开题答辩

答辩人: XXX 导师: XXX 2026 年 2 月

1. 研究背景 — 编译器软件防御机制

编译器为用户代码生成的软件防御机制，是系统安全的关键防线

主流编译器防御机制 (DeFuzz 覆盖面)

Stack Canary

栈保护 canary, 检测缓冲区溢出覆盖返回地址

FORTIFY_SOURCE

对 memcpy / strcpy / strcat 等注入 _chk 变体做边界检查

CET-IBT / BTI / PAC

控制流完整性: 间接跳转 landing pad + 指针鉴权

SafeStack / ShadowStack / StackClash

栈硬化: 分离敏感栈 + 探针防跨越

问题: 静默的逻辑漏洞

无崩溃 · 无报错

防御机制失效时程序表面行为正常, 漏洞潜伏在 compiler backend 与 libc runtime 之间

传统 crash-oracle 失效

AFL / libFuzzer 依赖段错误、assertion、ASan 报错, 对 ' 编译器没按规范生成防护代码 ' 无感

安全影响巨大

单个 bug 可使整类二进制的安全保证沦为纸糊, 典型代价参见 CVE-2023-4039 (AArch64 canary 绕过)

2. 研究现状 — LLM 驱动的编译器 Fuzzing

近年 LLM 引入编译器 fuzzing, 但聚焦优化正确性, 未覆盖软件防御机制

WhiteFox (OSDI'24)

LLM 引导路径触发

- 用 LLM 阅读优化 pass 源码, 生成触发特定优化的 C 程序
- 擅长 miscompile 等 observable bug
- 依赖手工编写优化规则, 覆盖面有限

HLPFuzz (ICSE'25)

LLM 变异 + 覆盖率引导

- 把 LLM 作为变异算子与覆盖率 fuzzer 混合
- 聚焦语义 bug (crash / miscompile), 以 crash oracle 为主
- 缺少 invariant 级别的防御机制语义契约

共同局限

现有工作均无针对 '软件防御机制是否被编译器正确生成并维持有效' 的形式化 oracle —— 这恰是编译器安全链路上最脆弱、也最难自动化检测的环节。

3. 研究动机 — 编译器自身的防御实现仍在出错

真实案例：FORTIFY_SOURCE 在 GCC 的优化路径上静默失效

FORTIFY_SOURCE 把 strcpy / strcat 等函数替换成 `__*_chk` 变体，运行时比对目标缓冲区容量与写入长度——一旦 GCC 改写错了 `_chk` 参数，这层保护就默默失效。

GCC 对 strcat 调用的改写链 (FORTIFY_SOURCE=2)



第三个参数 `objsize` 应为 `objsize - strlen(dst)` —— 优化路径没有调整，`bound` 仍按原缓冲区大小，保护静默退化。

上游状态

已被 GCC 确认并 patch 进 master 分支，gcc 16.1+ 已修复；缺陷自 gcc 4.7 起存在约 14 年，社区计划 backport 至 gcc 13 及以上。

4. 研究目标与内容

研究目标 (Goals)

G1 · 抽取不变量

系统化提取各防御机制的 *invariants*

- 来源：源码注释 · 官方文档 · 已确认历史 bug
- 24 份 invariant survey 作为形式化依据

G2 · 构建预言机

把每条 *invariant* 落成可执行 *Checker*

- 聚合为 MechanismOracle (多阶段调度)
- 跨 ISA 统一接口, 支持正 / 负控

G3 · 覆盖率驱动 Fuzzing

以防御机制源码覆盖率为核心信号

- CFG-guided target 选择 + LLM 种子变异
- 发现静默逻辑漏洞 (GCC / LLVM)

核心挑战 (Challenges)

C1 · 测试预言机缺失

防御失效是 '静默' 逻辑漏洞 —— 无 crash、无报错; 传统 crash-oracle fuzzer 全盘失效。

C2 · 深层路径难触发

GCC 千万行代码, 防御逻辑分散多 pass; 随机变异难以生成满足路径约束的 seed。

思路起源: CVE-2023-4039 (AArch64 canary 绕过) 启发我们系统化抽取防御机制的不变量。

5. 研究方案 — 三步骤 workflow

Invariants 调研 → Oracle 落地 → LLM + 覆盖率驱动 Fuzzing



Fuzzing 主循环

target → prompt → LLM → compile + gcov → execute (native / QEMU) → MechanismOracle → coverage / corpus / retry 反馈

5.1 技术贡献

支撑上述三步骤的三项原创工程贡献

T1 · 覆盖率插桩 GCC 工具链

- 多 ISA 交叉编译: x64 / aarch64 / loongarch64 / riscv64
- gcovr 读取 .gcda 构建防御机制源码覆盖率
- QEMU-user 运行交叉产物, 统一退出码协议
- 与主循环异步: 每 seed 独立产物, 支持断点恢复

T2 · 多 invariant Oracle 框架

- InvariantChecker 接口 (Enablement / Static / Dynamic)
- MechanismOracle 三阶段调度, 早退化早报错
- Polarity-aware: -fno-* 负控下自动翻极性
- Finding 带 invariant ID + 一手证据链接

T3 · Contract + 分层 Prompt

- 三段式 prompt: base + understanding + contract
- RequiredMarkers 后置校验 LLM 响应
- Strategy ↔ Mechanism ↔ Oracle 绑定, 防错配
- FlagProfile 注入 CFLAGS 变种, 驱动 negative control

6. Oracle 实例 — Stack Canary 二分检测

4 个 Invariant Checker 覆盖 canary 机制的正确性契约

核心原理

渐进增加 buffer 填充 → 观察 ret / canary / buf 三者位置的单调退出状态变化 → 二分定位最小崩溃长度。

已注册 InvariantChecker (4)

INV-SP-G01	Static	__stack_chk_fail 符号存在
INV-SP-A01	Static	main 不应持有 canary 槽
INV-SP-L01	Dynamic	buffer 溢出二分搜索 + 哨兵
INV-SP-R03	Dynamic	epilogue 必须 scrub canary 寄存器 (新)

退出状态判定表 (INV-SP-L01)

stdout	exit	判定
有 SEED_RETURNED	139	Canary 绕过 (BUG)
无	139	间接崩溃 (假阳性)
—	134	Canary 生效 (安全)

哨兵 & 跨架构执行

seed() 返回前打印 "SEED_RETURNED" —— 区分 '真 canary 绕过' 与 '函数内部间接崩溃'。

0x41 填充在 x64 / aarch64 / loongarch64 / riscv64 均映射到未映射地址，稳定触发 SIGSEGV。

交叉编译 + QEMU-user 执行，退出码统一由主循环判定。

7. 初步实验结果

已在 GCC 上覆盖多防御机制 × 多 ISA, 发现 3 个真实漏洞

防御机制	x64	aarch64	loongarch64	riscv64
Stack Canary	✓	✓	✓	✓
FORTIFY_SOURCE	✓	—	—	—
CET-IBT	✓	N/A	N/A	N/A

已发现 3 个 GCC 防御实现漏洞 (均已上报, 其中 1 个已进 master)

Bug 1 · Canary epilogue 残留

长尾 ISA (loongarch64 / mips / xtensa / csky) 的 generic fallback 未在 ret 前 scrub caller-saved 寄存器, guard 值泄露。

Bug 2 · FORTIFY bound 未更新

tree-ssa-strlen 把 __strcat_chk 改写为 __strcpy_chk 时, arg3 (objsize) 仍按原缓冲区大小传递, 保护静默退化。

Bug 3 · CET-IBT 谓词不充分

movabsq 立即数若在非最低字节位置包含 ENDBR64 字节序列, 可构造出编译器未预期的 landing pad, 削弱 IBT。

后续实验

扩展至 LLVM 工具链 + 补全 GCC 其他防御机制 (SafeStack · CFI · PAC · BTI · ShadowStack · StackClash 等)。

8. 预期成果

学术产出 + 工程产出 + 社区影响

学术产出

- 1 篇 CCF-A / B 级会议或期刊论文 (目标方向: 安全 / 系统)
- 开源 DeFuzz 工具 (Go + Python, 多 ISA 支持)
- LLM-driven 编译器防御机制测试方法论
- 跨机制 invariant survey 作为独立贡献 (24 份)

工程 & 社区

- 向 GCC / LLVM 上游报告真实防御实现漏洞
- 支持 GCC + LLVM 双工具链, 便于回归
- 可扩展的 Invariant Oracle 体系, 覆盖面持续增长
- 自动化流水线 (插桩编译 / QEMU 执行 / 报告)

9. 进度安排

当前节点 → 毕业答辩的时间线





感谢各位老师的聆听！

敬请批评指正